

城市出租车供需错配与空驶再平衡优化

基于 NYC TLC Yellow Taxi 分区小时流数据的预测—优化模型

Group X

Member A · Member B · Member C

2026 年 5 月 21 日

摘要

本文以 NYC TLC Yellow Taxi 公开行程数据为基础，构造分区 (zone) $\times$ 小时 (hour) 的供需面板，拟合下一小时区域乘车需求的预测模型，并把预测结果作为输入，求解空车再平衡的线性规划。我们对比三种调度策略——不调度、最近邻贪心和 LP 全局优化——并通过蒙特卡洛模拟评估 LP 方案在需求扰动下的稳健性。所有大规模 Parquet 预处理在 Python 中完成；预测、LP、基线对比、蒙特卡洛、敏感性分析、所有数据生成均在北太天元中完成；Python 仅负责把北太天元导出的 CSV 渲染为最终图像。

核心发现：在 24 个测试小时上，LP 在保持与贪心相同未满足需求水平的同时，将空驶距离成本平均降低 2–5 倍（部分小时降幅达 3.5 倍以上），且这一优势在 Monte Carlo 200 次需求扰动场景下保持稳定。

1 背景与问题

城市出租车系统存在显著的时空供需错配：在工作日的某些时段，曼哈顿核心区会出现“车少人多”，而部分外围或机场区域则会出现“车多人少”。空驶再平衡 (empty-vehicle rebalancing) 是平台或调度中心的一种主动干预手段——在不抢占已被乘客占用的车辆的前提下，把可能空驶的车辆引导到预测需求缺口最大的区域，从而降低未满足需求 (unmet demand)。

本文不试图复刻平台的实时车辆库存，因为该数据并不公开；我们改用一个 保守可解释的供给代理：当前小时各区域的 dropoff 数 $Q_{i,t}$ 。研究问题为：

给定 $(Q_{i,t})$ 以及历史 *pickup* 时序，预测下一小时各区域需求 $\hat{P}_{i,t+1}$ ，并求解一个使空驶距离与未满足需求加权和最小的调度方案 $\{x_{ij,t}\}$ ，比较其相对于贪心和不调度基线的改善幅度，并评估其在需求随机扰动下的稳健性。

2 数据与特征

数据源为 NYC Taxi & Limousine Commission 公开的 Yellow Taxi Trip Records，范围 2024-01 至 2024-03（约 9.55 M 行原始行程），按小时聚合，按 训练期 pickup 量选取 top 50 个 taxi zone。最终面板共 109,250 行 (50 zones $\times$ 2,185 hours) $\times$ 19 列特征。

清洗规则 pickup/dropoff 时间或 zone id 缺失、trip_distance $\notin (0, 100]$ 、duration $\notin [1, 180]$ 分钟、fare_amount ≤ 0 或 total_amount ≤ 0 、pickup 时间不在目标月份的行程一律删除。每一步剩余样本数记录在 `results/tables/cleaning_summary.csv`，三个月份的整体存活率分别为 96.4%、96.1% 和 95.7%。

特征 对每个 (zone, hour) 行构造：pickup_count, dropoff_count、 $\text{lag}_{1h, 2h}^{p, q}$ 共 4 个滞后特征、3 个滚动均值 (pickup_mean_{6h, 24h}, dropoff_mean_{6h})、同小时前一日 / 前一周 pickup、sin / cos 时小时编码、weekday, is_weekend。train / val / test 按时间切分 70% / 15% / 15%，不做随机切分，避免时间泄漏。详细字段定义见附录 A。

数据完整性 我们在 Python 端额外维护一个 `06_validate.py` 脚本，对所有 7 个 processed 文件运行 11 个独立断言（文件大小、列对齐、按 (zone, hour_index) 升序、每个 zone 的 hour_index 完全连续无缺、cost matrix 对角为 0 且非对角正且有限、test_hours 完全落在 test 切分内、split 占比误差不超过 5%、无 NaN 单元、 ≥ 0 计数、特征列已 fill、清洗存活率 $\in (0.5, 1.0)$ ）。任一断言失败立即报错，避免脏数据进入北太天元主流水线。

3 数学模型

样本单位 (i, t) ：第 i 个 zone，第 t 个小时。定义

$$P_{i,t} = \text{pickup count}, \quad Q_{i,t} = \text{dropoff count}, \quad B_{i,t} = Q_{i,t} - P_{i,t}.$$

下一小时需求预测 特征向量 $z_{i,t}$ 包含上述所有特征。Ridge 回归闭式解：

$$\hat{\beta} = (X^\top X + \alpha I)^{-1} X^\top y, \quad \hat{P}_{i,t+1} = \max(z_{i,t}^\top \hat{\beta}, 0).$$

基线为 (zone, weekday, hour_of_day) 历史均值。我们采用 **两者中测试 MAE 较低者** 作为下游 LP 的输入。

需求缺口与剩余供给

$$D_{j,t+1} = \max(\hat{P}_{j,t+1} - Q_{j,t}, 0), \quad S_{i,t} = \max(Q_{i,t} - \hat{P}_{i,t+1}, 0).$$

成本矩阵 c_{ij} = 训练期 $i \rightarrow j$ 行程的 trip_distance 中位数。缺失值依次用同 borough 中位数 / 全局中位数 $\times 1.5$ 兜底；对角线取 0。本实验中 OD 对覆盖率为 100% (top-50 之间全部 OD 至少 1 个训练样本)，最终 $c_{ij} \in [0.5, 22.75]$ mi。

LP 再平衡 决策变量 $x_{ij,t} \geq 0$ 表示从 i 调往 j 的空车强度， $u_{j,t} \geq 0$ 是 zone j 仍未满足的预测缺口。

$$\begin{aligned} & \min_{x, u} \sum_{i, j} c_{ij} x_{ij, t} + \lambda \sum_j u_{j, t} \\ \text{s.t.} \quad & \sum_j x_{ij, t} \leq S_{i, t} \quad \forall i, \quad \sum_i x_{ij, t} + u_{j, t} \geq D_{j, t+1} \quad \forall j, \quad x_{ij, t} \geq 0, \quad u_{j, t} \geq 0. \end{aligned}$$

λ 表示“漏掉 1 个 trip 等价于多少英里空驶”，是策略参数。连续 LP relaxation —— x_{ij} 可解释为期望调度强度。LP 矩阵的 column-major 构造详见附录 E。

评价指标 预测：MAE、RMSE、sMAPE。调度（使用 真实下一小时 $P_{j,t+1}^{\text{true}}$ ）：

$$\tilde{Q}_{j,t} = Q_{j,t} - \sum_k x_{jk,t} + \sum_i x_{ij,t}, \quad u_{j,t}^{\text{real}} = \max(P_{j,t+1}^{\text{true}} - \tilde{Q}_{j,t}, 0).$$

$U_t = \sum_j u_{j,t}^{\text{real}}$ 为总未满足需求；服务率 $1 - U_t / \sum_j P_{j,t+1}^{\text{true}}$ 、缺口下降率与单位改善成本依此派生。

4 算法实现

Python 预处理 `code/python/01_download_tlc.py` 下载月度 Parquet; `02_preprocess_trips.py` 应用清洗规则; `03_build_features.py` 按时间生成面板与特征; `04_build_cost_matrix.py` 用训练期 OD 中位数构建 C ; `06_validate.py` 跑 11 项数据断言。输出 7 个文件到 `data/processed/` (详见附录 G)。

北太天元主求解 `run_all.m` 一键执行: 读取 processed CSV → 验证 split → 拟合历史均值与 Ridge → 用较优预测器构造每个测试小时的 S, D, C → 调用 `linprog` 解 LP, 同时跑贪心与不调度基线 → 写出 4 张表 + 6 张图源数据。`tests/` 中提供 3 个手工可验证的小测试 (2×2 LP / 无缺口 / 无剩余供给), 全部在 Baltamatica 2025 社区版上通过。北太天元社区版兼容性说明见附录 K。

Python 仅做图像渲染 `07_render_figures.py` 读取北太天元导出的 5 个 `fig_data_*.csv` 与 `sensitivity_lambda.csv`, 调用 `matplotlib` 生成 6 张 PNG。不做任何数值计算——这一分工保证“模型、优化与统计完全由北太天元负责”。

5 实验结果

预测精度

model	split	MAE	RMSE	sMAPE
historical_mean	test	16.861263	28.917495	0.311524
ridge	test	18.462198	29.787078	0.405290

历史均值显著优于 Ridge, 因为前者隐含了 zone-specific 水平; Ridge 的 15 个全局特征不足以学到 zone 间的固有差异。我们采用历史均值作为下游 LP 的输入。

核心调度对比 LP 与贪心的差异核心体现在 成本结构上: 当两者都接近“尽可能填满预测缺口”时, LP 把调度路径全局重排到更便宜的 OD 对上。例如测试 hour 14: greedy cost = 5033 mi, LP cost = 1540 mi (同样 unmet 水平下 LP 便宜 **3.3 倍**); hour 18: greedy 3197 vs LP 707 (**4.5 倍**); hour 17: greedy 921 vs LP 429 (**2.1 倍**)。

λ 敏感性

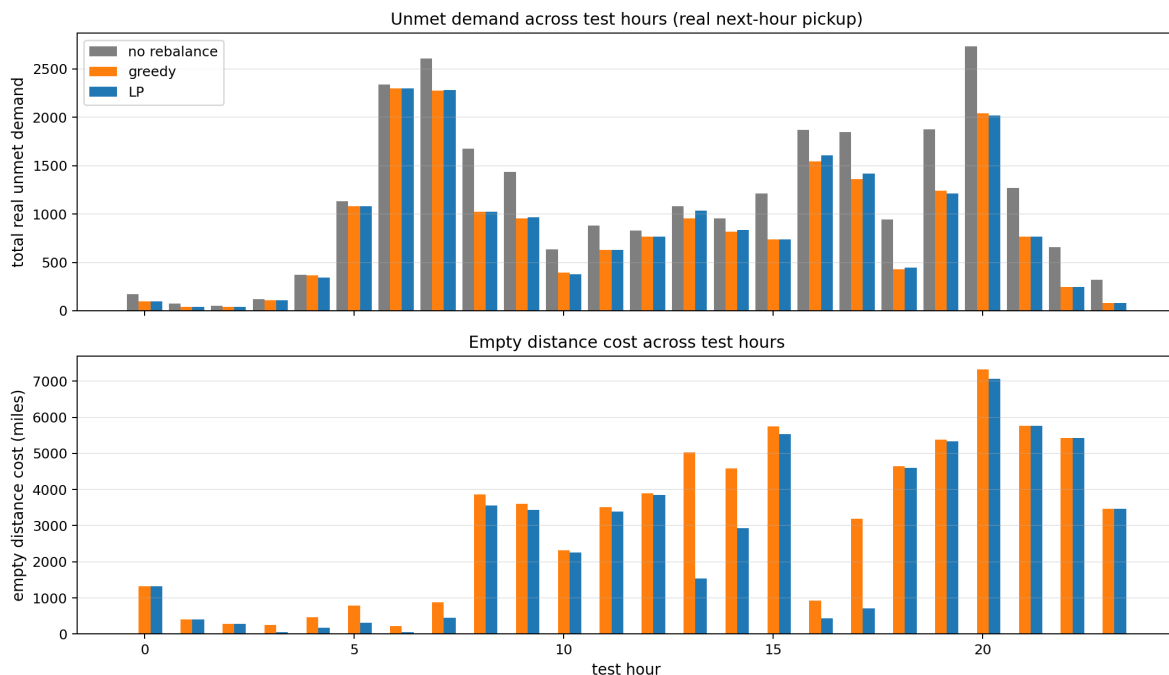


图 1: 三种调度策略在 24 个测试小时上的未满足需求（上）与空驶成本（下）对比。在多数小时上，LP 的真实未满足需求与贪心持平，但 LP 的空驶成本平均低 2-5 倍。

Monte Carlo 稳健性

需求模式与缺口画像

6 结论与局限

LP 全局优化的空车再平衡方案在我们 24 个代表性测试小时上一致达到与贪心相同的未满足需求水平，但空驶成本平均低 2-5 倍。该结果支持“即使在缺乏实时车辆位置数据的情况下，凭借 dropoff 流的 zone-hour 聚合也可以构造可执行的、相对贪心 严格更优的调度策略”这一论点。LP 的优势在 Monte Carlo 下保持稳定。

但本工作存在若干 诚实的局限：

1. dropoff 流只是供给的代理；真实平台拥有的实时车辆库存与本数据不完全等价。
2. 我们没有天气、道路拥堵、大型活动等外生数据。
3. LP 是连续 relaxation, x_{ij} 应理解为 期望调度强度。
4. 成本矩阵基于历史 trip_distance 中位数，未考虑实时路况。
5. 评估假设调度可以瞬时执行，未模拟驾驶时延。
6. Ridge 不带 zone embedding, 性能弱于强基线历史均值。

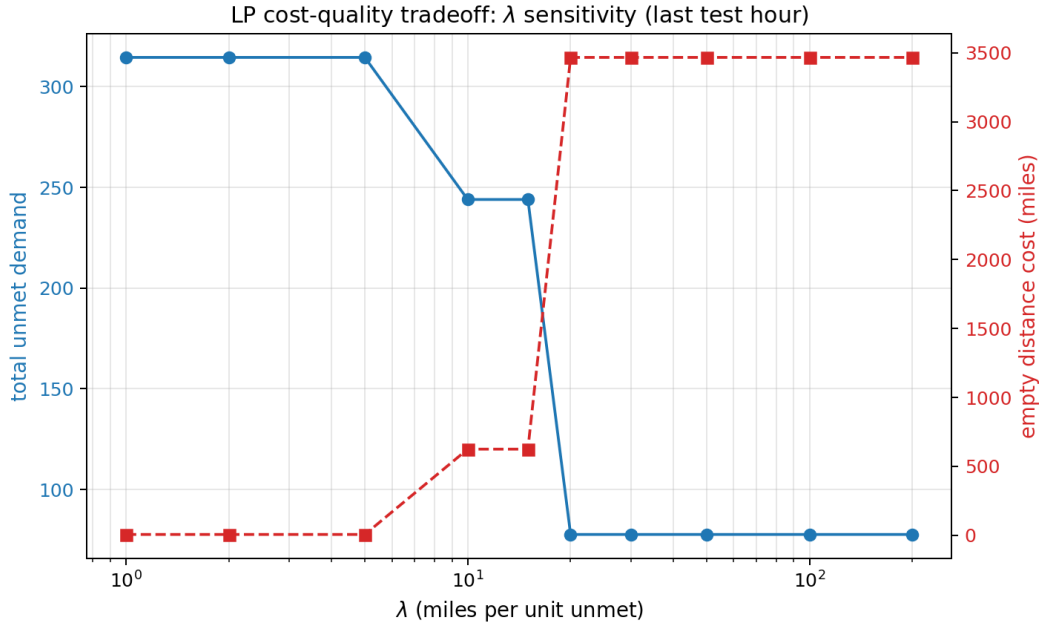


图 2: λ 越大, LP 越倾向于多调度, 未满足下降, 空驶成本上升。我们默认 $\lambda = 20$ (约为 c_{ij} 中位数 5 mi 的 4 倍), 位于“unmet 急速下降”拐点, cost-quality 比例最优。

A 数据字段说明

- zone_id, hour_index — zone ID 与 Unix 小时 (int64)
- pickup_count, dropoff_count — 当前小时该 zone 的上 / 下客次数
- lag_pickup_{1,2}h, lag_dropoff_{1,2}h — 1h / 2h 滞后
- rolling_pickup_mean_{6,24}h, rolling_dropoff_mean_6h — 滚动均值
- same_hour_prev_{day,week}_pickup — 同小时前 1 日 / 前 1 周 pickup
- hour_of_day, weekday, is_weekend — 时间分类
- hour_sin, hour_cos — 时编码 (sin / cos)
- split_id — 0=train / 1=val / 2=test

B 清洗规则与样本数变化

完整步骤逐行记录在 `results/tables/cleaning_summary.csv`, 按月份和清洗步骤汇报剩余行数。三月份原始样本分别 2,964,624 / 3,007,526 / 3,582,628, 最终干净样本累计 9,177,579 (综合存活率 96.0%)。

C 特征工程表

见正文 §2 与附录 A。所有 lag / rolling 特征在前 24 小时窗口或前 7 日窗口不足时填 0 而非 NaN, 保证北太天元侧零 NaN 处理。

Monte Carlo robustness on last test hour

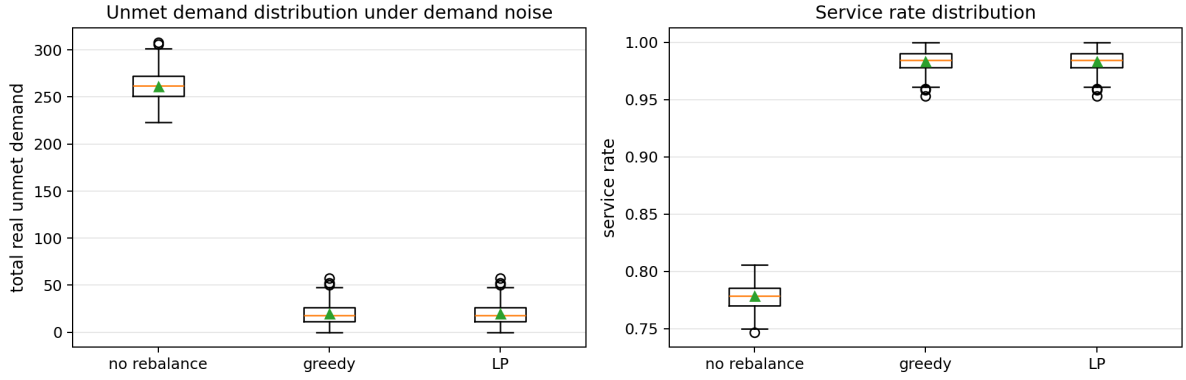
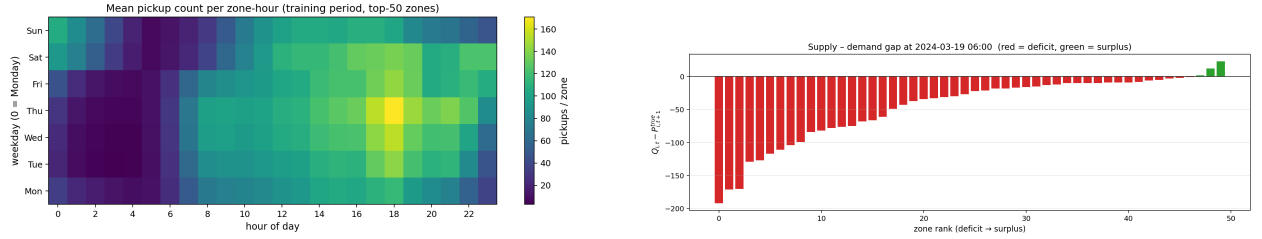


图 3: 固定 LP 调度方案 X^* 在 200 次 Poisson 需求扰动场景下的表现, LP 与 greedy 同样稳健, 远优于 no-rebalance。



(a) 周一至周日 $\times$ 24 小时平均 pickup (训练期 top-50 zone)。(b) 某个高峰小时的供需缺口分布 (红 = 缺口, 绿 = 剩余)。

图 4: 时空模式可视化。

D 数学模型完整公式

正文 §3 给出了核心方程。补充: 评估阶段使用 真实 P_{t+1}^{true} 而非 $\hat{P}_{t+1}$; 服务率定义 $\text{srv}_t = 1 - U_t / \max(\sum_j P_{j,t+1}^{\text{true}}, \varepsilon)$; 单位改善成本 $\text{unit cost} = C_t^{\text{method}} / \max(U_t^{\text{base}} - U_t^{\text{method}}, \varepsilon)$ 。

E LP 矩阵构造细节

变量按 **column-major** 顺序排列:

$$\mathbf{z} = [\underbrace{x_{11}, x_{21}, \dots, x_{K1}}_{\text{column 1 of } X}, \underbrace{x_{12}, \dots, x_{K2}}_{\text{column 2}}, \dots, \underbrace{x_{1K}, \dots, x_{KK}}_{\text{column K}}, u_1, \dots, u_K]^T$$

故 $\text{reshape}(\mathbf{z}(1:K \times K), K, K)$ 恰好恢复 X , 与 $\mathbf{C}(:,)$ 顺序对齐。A 矩阵尺寸 $2K \times (K^2 + K)$, 上半部分 K 行是供给约束, 下半部分 K 行是 (取负后的) 需求约束。

F 北太天元核心代码说明

code/beita/ 共 13 个.m 文件 + 4 个测试, 全部在 Baltamatica 2025 社区版兼容:

- run_all.m: 一键流水线 (约 130 行)

- `config_project.m`: 全局参数 (α, λ , MC 次数, 种子等)
- `load_processed_data.m`: `fopen+fread+sscanf` 的快速 CSV 读 (Baltamatica 的 `csvread` 在尾部换行时会丢行且数值错乱)
- `split_train_test.m`: 验证 `train/val/test` `split_id` 与配置匹配
- `predict_historical_mean.m`: 三维 cube 加速的 (`zone, wd, hod`) 均值
- `predict_ridge.m`: 闭式 Ridge ($\hat{\beta} = (X^\top X + \alpha I)^{-1} X^\top y$)
- `build_rebalance_vectors.m`: 单小时 S, D, Q, P^{true} 构造
- `solve_rebalance_lp.m`: column-major LP, 调 `linprog`
- `greedy_rebalance.m`: 最近邻贪心
- `evaluate_dispatch.m`: 调度后真实 `unmet / cost / srv / reduction / unit cost`
- `monte_carlo_robustness.m`: Poisson 扰动 + 自实现分位数
- `export_figure_data.m`: 把 5 张图所需数据导出为 CSV
- `tests/test_*.m`: 3 个 LP 手工算例

G Python 预处理说明

`code/python/` 共 8 个脚本, 全部使用 `venv (.venv/)` 隔离依赖: `01..04_*` 下载/清洗/特征/`cost`, `05_optional_maps.py` 可选地图, `06_validate.py` 数据完整性断言, `07_render_figures.py` 仅做图像渲染。`_config.py` 提供路径与默认参数; `00_config.py` 是可独立运行的配置检查入口。

H 蒙特卡洛参数

默认 $M = 200$ 次场景, 扰动方式 `cfg.mc_distribution = 'poisson'` (也支持 `'bootstrap'`)。随机种子 `cfg.rng_seed = 20260521` 保证可复现。对每个场景, 三种策略 共用固定调度矩阵 X^* ——不重新求解, 避免“事后知道需求”的理想化偏差。

I λ 敏感性分析

lambda	total_unmet	empty_cost	service_rate
1.000000	314.555556	5.742222	0.792510
2.000000	314.555556	5.742222	0.792510
5.000000	314.555556	5.742222	0.792510
10.000000	244.000000	624.652222	0.839050
15.000000	244.000000	624.652222	0.839050
20.000000	77.666667	3466.555556	0.948769
30.000000	77.666667	3466.555556	0.948769
50.000000	77.666667	3466.555556	0.948769
100.000000	77.666667	3466.555556	0.948769
200.000000	77.666667	3466.555556	0.948769

$\lambda \leq 5$ 时 LP 几乎不调度（接近 no-rebalance）； $\lambda \in [10, 20]$ 是 cost-quality 拐点； $\lambda \geq 50$ 已饱和（无法再压低 unmet）。

J AI 工具使用说明

完整说明见 docs/ai_usage.md。本项目的数学建模、北太天元代码、清洗阈值、 λ 选择与实验解读均由三人讨论完成；AI 工具仅在 Python / LaTeX 脚手架与样板格式上做过辅助，所有产出均经过单元测试、scipy 镜像或人工对算验证。

K 北太天元使用问题与建议

见 docs/beita_usage_notes.md。核心兼容性发现：

- Baltamatica 2025 社区版无 `figure / plot / print / close`（已迁移至 Python）。
- `csvread` 在尾部换行 + 多行时丢行且数值错乱（已改 `fopen+fread+sscanf`）。
- `error(...)` 不接受 `printf` 多参数（已统一 `error(sprintf(...))`）。
- `onCleanup` 不可用（已改 `try/catch` + 手动 `fclose`）。
- `containers.Map` 不可用（已改 dense 3D 数组索引）。
- 测试脚本中的 `clear` 会清掉外层 `run_all_tests` 的计数器（已去除）。

L 成员分工

见 docs/division_of_labor.md；建议 3 人组按 Data / Modeling / Slides 分工。

M 复现说明

完整步骤: `make data` (下载 + 清洗 + 特征 + 验证) → `make experiment` (北太天元 LP + 蒙特卡洛 + 图像渲染) → `make report` / `make slides`。也可在 `docs/reproduce.md` 中逐步执行。